Contexto completo del proyecto Mascotas Municipales

Documento de transferencia para integrantes y asistentes de desarrollo

Este documento reúne el contexto funcional, técnico y académico de la aplicación Android Mascotas Municipales. Su propósito es que un compañero o un nuevo chat pueda entender qué problema se está trabajando, qué funciones ya existen, cómo está organizado el código, qué se ha probado, cuáles son los límites actuales y cuál es el siguiente trabajo solicitado.

Dato	Información
Aplicación	Mascotas Municipales
Contexto	Prototipo académico Android para Zipaquirá, Cundinamarca
Versión revisada	1.5 - versionCode 6
Asignatura	Desarrollo Móvil
Profesor	Wilber Hernando García Gaitán
Fecha de actualización	2 de octubre de 2026
Repositorio local	D:\MascotasMunicipalesNative_Corregido

Integrantes

- Joseph Trout Rodriguez
- Juan David Campuzano
- Ricardo Avila
- Juan Pablo Cerquera Ramirez
- Daniel Mateo Segura Briceno

Estado general. La aplicación es un prototipo funcional y demostrable, pero no es un sistema oficial de la Alcaldía de Zipaquirá. Debe utilizar datos y cuentas ficticias.

1 Resumen para entender el proyecto

Mascotas Municipales busca reunir en una sola aplicación el registro de mascotas, los reportes de pérdida o hallazgo, la consulta pública por QR y la coordinación básica entre ciudadanía, veterinarios y administración municipal. La aplicación está desarrollada en Kotlin como una app Android nativa y conserva un diseño en español con color turquesa.

El ciudadano puede crear una cuenta, registrar mascotas, consultar fichas públicas, crear reportes y ver su seguimiento. Un veterinario aprobado puede consultar los casos asignados, registrar una atención y proponer jornadas territoriales. Un administrador aprobado revisa reportes, asigna casos, cierra procesos y aprueba o rechaza eventos. Solo los eventos futuros aprobados aparecen en Territorio.

Firebase Authentication maneja las cuentas y Cloud Firestore almacena mascotas, reportes, permisos, casos, registros clínicos y eventos. La app permanece en el plan Spark y no utiliza Firebase Storage, Functions ni servicios pagos. Las fotografías actuales son recursos locales de demostración.

Funciones principales actuales

- Registro e inicio de sesión con correo y contraseña.
- Directorio de mascotas con búsqueda local, ficha pública y registro de nuevas mascotas.
- Reportes de pérdida o hallazgo con filtros, detalle, estado y seguimiento.
- Consulta de mascota por lector QR o código escrito manualmente.
- Mapa de Zipaquirá con OpenStreetMap y eventos territoriales aprobados.
- Flujos separados para ciudadano, veterinario y administrador.
- Accesibilidad con texto ampliado, alto contraste, paleta azul para daltonismo y etiquetas para TalkBack.
- Borradores locales y recuperación de envíos ciudadanos con un identificador estable.
- Mensajes diferenciados para caché, envío pendiente, confirmación del servidor y error reintentable.

Qué no debe asumirse

- No es una plataforma oficial ni contiene un censo municipal real.
- No presta servicios veterinarios ni realiza vacunaciones; los eventos son anuncios ficticios.
- No sube fotografías del dispositivo a Firebase.
- No permite que una cuenta se convierta en veterinaria o administradora desde la aplicación.
- Las reglas e índices más recientes de roles y eventos todavía deben verificarse y desplegarse en Firebase remoto con autorización.

2 Problema objetivo y fundamento de investigación

El problema principal es que la información de mascotas perdidas, encontradas o atendidas suele quedar repartida entre publicaciones, mensajes y registros diferentes. Esto dificulta identificar al animal, saber dónde fue visto, conservar el historial del caso y coordinar a las personas que deben actuar.

El objetivo del prototipo es organizar esa información en una aplicación móvil accesible, con permisos por rol, funcionamiento parcial sin conexión, consulta pública limitada y herramientas de seguimiento. El enfoque territorial está centrado en Zipaquirá y todavía usa cuatro comunas simplificadas.

Hallazgos que orientaron el diseño

Hallazgo	Implicación para la aplicación
MinSalud estimó 23.596 perros y gatos en Zipaquirá para 2024.	Una solución real debe pensar en decenas de miles de animales y un histórico mayor de reportes y eventos.
Una fuente oficial de INVIMA basada en proyección DANE reporta 174.415 habitantes para 2026.	La escala municipal supera una demostración pequeña y exige consultas acotadas, paginación y pruebas de carga.
El CNPV 2018 registró dificultades para actividades diarias en 8,7 % de la población censada.	La accesibilidad no es un adorno: se requieren TalkBack, texto escalable, contraste, mensajes claros y controles amplios.
La investigación encontró referencias a 52 barrios en 4 comunas, además de corregimientos, centros poblados y veredas.	El catálogo territorial productivo debe venir de Planeación Municipal, estar versionado y no quedar codificado como un número fijo.
La conectividad puede ser variable y la información incluye datos personales.	Se necesitan recuperación local, estados claros, minimización de datos, privacidad desde el diseño y control de acceso.

Criterios legales y de privacidad considerados

- Ley 1581 de 2012 y lineamientos de la SIC sobre protección de datos y privacidad desde el diseño.
- Ley 1618 de 2013 y Ley 1346 de 2009 para accesibilidad y diseño universal.
- Ley 1774 de 2016 sobre animales como seres sintientes y normas relacionadas con convivencia y bienestar animal.
- El QR no debe contener directamente nombre, teléfono, documento o dirección del responsable.
- Una implementación real requiere responsables institucionales, retención definida, auditoría y procedimientos de actualización o supresión de datos.

3 Evolución registrada en Git

El historial de Git muestra una evolución incremental. Primero se creó la base Android, después se conectó Firebase y luego se añadieron mejoras de estabilidad, accesibilidad, recuperación, roles, eventos, diseño, mapa y QR.

Versión o cambio	Qué se incorporó
Inicio	Proyecto Android nativo, navegación inicial, pantallas y recursos de mascotas de demostración.
Firebase	Authentication, Cloud Firestore, reglas, índices, emuladores y pruebas de seguridad.
Conexión y rendimiento	Consultas limitadas, mensajes de caché, menos lecturas y protección frente a doble envío.
Accesibilidad	Texto 30 % más grande, alto contraste, paleta azul, etiquetas TalkBack y acceso a ajustes de Android.
Versión 1.2	Borradores por cuenta, guardado atómico local, ID estable y recuperación de envíos tras reiniciar.
Versión 1.3	Acceso profesional, revisión de reportes, asignación a veterinario, atención y cierre de casos.
Interfaz	Jerarquía visual común, tarjetas, cápsulas de estado, navegación con iconos y animaciones breves.
Versión 1.4	Propuesta veterinaria de eventos, revisión administrativa y publicación de eventos aprobados.
Mapa	OpenStreetMap mediante osmdroid, centro en Zipaquirá, atribución y caché controlada.
Versión 1.5	Icono, splash, información de la app, búsqueda, filtros, reintentos y consulta QR con alternativa manual.

Commits principales revisados

- 10dcedc - Initial commit
- f558bd6 - Implement Firebase Auth and Firestore prototype
- b3f5016 - Reduce Firestore reads and harden offline UI
- 6e75fbc - Accesibilidad ajustes
- db99f07 - Preserve drafts and recover Firebase submissions in version 1.2
- 04d4f4a - Add municipal admin and veterinary workflow in version 1.3
- 34273dc - Frontend
- 2678789 - Eventos
- 98e3963 - Mapa
- 4764e19 - Mejoras leves y versión 1.5

4 Usuarios roles y recorridos

Rol	Funciones	Restricciones
Ciudadano	Registrarse, iniciar sesión, registrar mascotas, crear reportes, buscar fichas, consultar QR, ver territorio y consultar el resultado de su caso.	No puede aprobarse como profesional, ver notas clínicas privadas ni administrar casos ajenos.
Veterinario	Consultar sus casos asignados, registrar valoración, atención y seguimiento, y proponer eventos territoriales.	Necesita access/{uid} activo; no puede ver todos los reportes ni aprobar su propio evento.
Administrador	Revisar reportes, crear casos, consultar equipo, asignar o reasignar veterinario, cerrar casos y aprobar o rechazar eventos.	No puede leer los registros clínicos privados del veterinario y no debe incluir información clínica en el resultado ciudadano.

Recorrido ciudadano

1. Crear una cuenta o iniciar sesión. La app prepara el documento de perfil ciudadano.

2. Abrir Mascotas para buscar, revisar fichas o registrar un animal.

3. Abrir Reportes para registrar una pérdida o hallazgo y consultar su estado.

4. Consultar el seguimiento operativo de un caso cuando la administración lo haya revisado.

5. Abrir Territorio para consultar el mapa y los eventos futuros aprobados.

6. Usar Perfil para accesibilidad, información de la aplicación o cierre de sesión.

Recorrido administrativo y veterinario

1. El administrador abre un reporte ciudadano y crea el caso con el mismo ID, evitando duplicarlo al repetir la acción.

2. El administrador asigna un veterinario activo de la misma dependencia.

3. El veterinario ve solo sus casos y registra valoración, atención y seguimiento. Las entradas son inmutables.

4. El administrador escribe un resultado operativo y cierra caso y reporte mediante una transacción.

5. El ciudadano consulta el estado y el resultado, pero no las notas clínicas.

Recorrido de eventos territoriales

1. El veterinario completa tipo, título, comuna, lugar, fecha, hora y descripción de una jornada futura.

2. La solicitud queda Pendiente y permanece privada para su autor y la administración.

3. El administrador aprueba o rechaza una sola vez; durante la decisión no se modifica el contenido.

4. Solo un evento Aprobado y futuro aparece públicamente en Territorio.

5 Funciones visibles de la aplicación

Área	Qué muestra o permite
Inicio	Resumen del prototipo, accesos principales, indicadores acotados y navegación inferior.
Mascotas	Listado de hasta 30 fichas, búsqueda local por nombre, especie, raza, comuna o código, registro y detalle.
Reportes	Listado propio de hasta 30 reportes, filtros Todos/Abiertos/Resueltos, creación, detalle y cambio a Resuelto cuando corresponde.
QR	Lectura mediante Google Code Scanner o escritura manual. Consulta exacta con límite de un documento y muestra solo datos públicos.
Territorio	Mapa de Zipaquirá con OpenStreetMap y hasta 20 eventos futuros aprobados ordenados por fecha.
Acceso profesional	Ingreso al espacio aprobado, listas paginadas de 20, casos, equipo veterinario y solicitudes de eventos.
Perfil	Accesibilidad, información de la aplicación y cerrar sesión. Cambiar correo, contraseña y datos de mascotas siguen como Próximamente.

Mapa y territorio

El mapa está construido con osmdroid 6.1.20 y teselas estándar de OpenStreetMap. Se centra en la referencia 5.021476, -73.990955, limita el zoom entre 11 y 18 y muestra atribución. No pide ubicación del dispositivo ni requiere clave de pago. La caché temporal tiene un máximo aproximado de 24 MiB y se recorta a 20 MiB. No existe descarga masiva ni un modo de mapa completamente offline.

Interfaz

La interfaz está creada con vistas nativas de Android de forma programática. Mantiene el turquesa original, encabezados con gradiente sutil, tarjetas con bordes y elevación ligera, estados en cápsulas y botones con respuesta táctil. Las transiciones usan una sola capa durante aproximadamente 180 ms para cuidar el rendimiento del emulador y respetan la escala de animación configurada en Android.

6 Arquitectura técnica y organización del código

Componente	Responsabilidad
MainActivity.kt	Construye las pantallas, navegación, formularios, mapa, QR, accesibilidad y recorridos ciudadanos y profesionales. Actualmente concentra gran parte de la interfaz.
AppViewModel.kt	Mantiene sesión, ruta, pestaña, borradores, solicitudes activas y recuperación aunque se recree la Activity.
MunicipalRepository.kt	Punto de acceso ciudadano a Firebase Authentication y Firestore: perfiles, mascotas, reportes, eventos públicos, conteos y confirmación de envíos.
WorkRepository.kt	Operaciones profesionales que exigen servidor: permisos, páginas de reportes/casos/equipo/eventos y transacciones de cambio de estado.
DraftStore.kt	Archivo privado por cuenta con AtomicFile dentro de noBackupFilesDir. No guarda contraseñas.
Models.kt	Modelos Pet, Report, Territory y MunicipalEvent, territorios de demostración y relación entre claves de foto y recursos locales.
firestore.rules	Reglas de autorización, validación de campos, propiedad, roles, estados y límites de consultas.
firestore.indexes.json	Índices compuestos para reportes, casos, equipo y eventos.
tests	Pruebas de reglas y protocolos manuales de recuperación, roles, eventos y demostración.
scripts	Siembra opcional y explícita de datos ficticios; nunca se ejecuta automáticamente desde la app.

Configuración técnica

Elemento	Valor actual
Lenguaje	Kotlin con interfaz Android nativa
Paquete	com.mascotasmunicipales
compileSdk	35
targetSdk	33
minSdk	24
Java	11
Gradle	9.5

Elemento	Valor actual
Android Gradle Plugin	9.3.1
Firebase BoM	34.19.0
Mapa	osmdroid 6.1.20 y OpenStreetMap
QR	Google Code Scanner 16.1.0

Decisiones de rendimiento

- Las búsquedas y filtros trabajan sobre los documentos ya cargados y no generan una consulta por cada tecla.
- Los directorios ciudadanos cargan hasta 30 documentos y los profesionales usan páginas de 20.
- Territorio hace una consulta puntual de hasta 20 eventos, no un listener permanente.
- Los conteos de inicio se limitan a 1.000 registros y se presentan como cifras acotadas.
- Las imágenes grandes de demostración se muestrean antes de colocarlas en pantalla.
- El mapa libera recursos cuando se abandona la pantalla.

7 Datos Firebase y privacidad

Ruta	Contenido y acceso
users/{uid}	Correo, rol ciudadano y fechas. Lectura propia. La contraseña permanece exclusivamente en Authentication.
access/{uid}	Rol admin o vet, dependencia, estado activo, nombre ficticio y aprobación. Solo una persona de confianza lo crea fuera de la app.
pets/{petId}	Ficha pública, ownerId opaco, clave de foto local, código QR y fechas. El alta es propia; los cambios requieren propiedad o autorización.
reports/{reportId}	Propietario, mascota opcional, tipo, especie, comuna, descripción, estado y fechas. Acceso del propietario y administración autorizada.
cases/{reportId}	Estado operativo, referencias, veterinario asignado, resultado ciudadano, versión e historial de cambios.
cases/{id}/clinicalRecords/{id}	Valoración, atención y seguimiento. Solo el veterinario asignado activo; registros inmutables.
cases/{id}/history/{version}	Actor, asignación, estado, versión y fecha. Registro técnico atómico.
events/{eventId}	Autor, dependencia, comuna, tipo, título, lugar, descripción, fecha, estado y revisión.
territories/{territoryId}	Comunas de demostración, actualmente comuna-1 a comuna-4.

Reglas de seguridad importantes

- Elegir Ciudadano, Veterinario o Administrador en la pantalla no concede permisos.
- Toda cuenta creada desde la app nace como ciudadana.
- El campo histórico users.role no concede acceso profesional; la autorización está en access/{uid}.
- Las reglas niegan por defecto rutas no declaradas, validan tipos y campos y bloquean escalada de privilegios.
- El ownerId es un UID opaco, no un correo, teléfono o dirección.
- La ficha pública y el QR no muestran información privada del responsable.
- Los datos clínicos no son visibles para el administrador ni para otros veterinarios.

Servicios y costos

El proyecto usa Firebase Authentication y Cloud Firestore en el plan Spark. No incorpora Storage, Functions, Analytics ni facturación. OpenStreetMap tampoco requiere una clave de pago. Las cuotas gratuitas no son ilimitadas: si se agotan, el servicio puede restringir operaciones.

8 Trabajo sin conexión borradores y recuperación

Firestore Android conserva caché y una cola local de escrituras. Además, la aplicación creó una capa propia para que los formularios ciudadanos no dependan únicamente de la Activity. Cada cuenta tiene un archivo privado de borradores y un borrador de mascota y reporte.

- El borrador se guarda aproximadamente 400 ms después de escribir, al navegar, al pasar a segundo plano y antes de enviar.
- El ID estable se crea y se escribe en disco antes de llamar a Firestore.
- Mientras el envío está pendiente, el formulario queda bloqueado para evitar duplicados.
- Reintentar conserva el mismo ID y el mismo contenido.
- Solo una confirmación de escritura o una lectura real del servidor marca el envío como confirmado; la caché sola no basta.
- Si el servidor rechaza o no confirma, el registro se conserva localmente y ofrece reintentar.
- Los borradores están separados por UID, no se sincronizan entre dispositivos y se eliminan al borrar los datos de la app o desinstalarla.

Diferencia importante

El comportamiento actual no es todavía el taller nuevo de sincronización manual. Firestore puede encolar una escritura cuando el usuario pulsa el botón del formulario y enviarla automáticamente cuando regresa la conexión. El taller nuevo exige que GUARDAR escriba solamente en una base local y que ningún dato llegue a Firebase RTDB hasta que el usuario pulse SINCRONIZAR.

9 Accesibilidad diseño y experiencia

Preferencias disponibles

- Texto de la aplicación 30 % más grande, adicional a la escala del sistema.
- Modo de alto contraste.
- Paleta azul para apoyar a personas con daltonismo.
- Acceso a los ajustes de accesibilidad de Android.

Criterios aplicados

- Estados expresados con palabras y no solamente con color.
- Pestaña seleccionada diferenciada por fondo y peso de texto.
- Etiquetas y descripciones para formularios, botones y registros compatibles con TalkBack.
- Campos agrupados, mensajes de error comprensibles y acciones de reintento.
- Animaciones cortas y simples para no volver lento el emulador.
- Preferencias guardadas únicamente en el dispositivo y no enviadas a Firebase.

Pendientes de accesibilidad

Todavía faltan pruebas manuales amplias con TalkBack, tamaños extremos de fuente y varios dispositivos físicos. La app no activa TalkBack por su cuenta, porque esa decisión pertenece al usuario y al sistema operativo.

10 Pruebas y evidencias existentes

Evidencia	Resultado documentado
Compilación	La versión 1.5 compiló correctamente con :app:assembleDebug y el APK fue actualizado.
Android Lint	Terminó con 0 errores bloqueantes y 44 advertencias conocidas en la versión 1.5.
Reglas Firestore	33 pruebas aprobadas y 0 fallidas en el emulador local.
Recuperación	Se verificaron borradores, reinicio del proceso, aislamiento por cuenta, envío sin red, reconexión y reintento con el mismo ID.
Roles	Se verificó el recorrido administrador a veterinario a ciudadano con 28 pruebas de reglas en la etapa 1.3.
Eventos	Se verificó propuesta, privacidad, aprobación, rechazo, publicación e inmutabilidad; la suite alcanzó 32 pruebas en la etapa 1.4.
QR	La prueba 33 comprueba consulta pública exacta sin acceso a perfiles. Existe alternativa manual cuando no hay cámara o Google Play Services.
APK	apk/MascotasMunicipalesNative-corregido.apk corresponde a la compilación debug de demostración 1.5.

Cuentas de demostración local

La siembra del emulador puede crear citizen@example.test, admin@example.test y vet@example.test. Son cuentas ficticias exclusivas del emulador local. La contraseña se suministra mediante la variable DEMO_PASSWORD y no debe guardarse en este documento, el repositorio ni el APK.

Comandos habituales

```
gradlew.bat :app:assembleDebug
```

```
gradlew.bat :app:lintDebug
```

```
firebase emulators:exec --only auth,firestore --project ID_REAL "node --test
tests/firestore.rules.test.mjs"
```

11 Límites actuales y trabajo pendiente

Tema	Estado o límite
Fotografías	Las fotos de mascotas son recursos incluidos en la aplicación. No existe carga de fotos del dispositivo ni Firebase Storage.
Perfil	Cambiar correo, contraseña e información de mascotas todavía aparece como Próximamente.
Territorio	Las cuatro comunas son una simplificación y deben validarse con Planeación Municipal.
Mapa	Necesita Internet para zonas no almacenadas recientemente y el servicio comunitario no garantiza disponibilidad permanente.
Módulo profesional	Exige servidor y transacciones en línea; sus borradores clínicos y de eventos no sobreviven al cierre completo del proceso.
Firebase remoto	Las reglas e índices más recientes deben revisarse y desplegarse juntos antes de usar roles profesionales remotos.
Pruebas físicas	Faltan pruebas amplias de cámara, TalkBack, varios dispositivos y concurrencia real.
Producción	No hay validación institucional, política de tratamiento definitiva, soporte, auditoría completa ni datos oficiales.

Próximas mejoras ya identificadas

- Completar opciones pendientes del perfil.
- Validar el catálogo territorial oficial y versionarlo.
- Mejorar pruebas TalkBack y compatibilidad en dispositivos físicos.
- Agregar historial visual, agenda o recordatorios sin comprometer Spark.
- Definir un proceso institucional real para aprobar profesionales y tratar datos.
- Separar progresivamente la interfaz concentrada en MainActivity en componentes más mantenibles.

12 Nuevo taller Offline First con Firebase RTDB

El requerimiento más reciente debe integrarse en el mismo proyecto, pero todavía no está implementado. Consiste en añadir un flujo de captura en campo que funcione completamente sin Internet y use Firebase Realtime Database como repositorio remoto de sincronización manual.

Condiciones mínimas del taller

- Grupos de 2 a 4 estudiantes. El equipo actual tiene cinco integrantes, por lo que este punto debe consultarse con el profesor.
- Cada registro debe tener mínimo 15 atributos, al menos 8 introducidos por el usuario y una imagen.
- GUARDAR debe almacenar únicamente en el dispositivo y nunca escribir en Firebase.
- La información y la imagen deben permanecer después de cerrar y volver a abrir la app.
- Solo el botón SINCRONIZAR puede enviar los registros pendientes a Firebase RTDB.
- Deben existir como mínimo los estados PENDIENTE y SINCRONIZADO.
- Una segunda sincronización no puede duplicar registros.
- Un fallo parcial debe dejar pendientes únicamente los registros no confirmados.

Propuesta para integrarlo en Mascotas Municipales

Puede crearse un módulo de censo o visita veterinaria en campo. El usuario captura los datos de la mascota, la ubicación territorial, información básica de la visita y una fotografía. Room Database guardaría los registros y el estado local; la imagen se copiaría al almacenamiento interno de la app; y un repositorio separado enviaría a RTDB únicamente cuando la persona pulse SINCRONIZAR.

Actualmente	Debe agregarse para el taller
Cloud Firestore con cola offline automática	Firebase Realtime Database con sincronización activada manualmente
DraftStore guarda un borrador de mascota y reporte	Room o equivalente para múltiples registros completos persistentes
Fotos locales incluidas como recursos	Imagen capturada o seleccionada y copiada al almacenamiento persistente
El botón del formulario llama a Firestore	GUARDAR solo crea un registro local PENDIENTE
Confirmación basada en metadatos de Firestore	Confirmación individual de RTDB y actualización local a SINCRONIZADO
ID estable para reintentos	Conservar el mismo UUID como llave de RTDB para evitar duplicados

Regla para no dañar lo existente

El módulo nuevo debe añadirse sin eliminar Authentication, Firestore, roles, eventos, QR, mapa, accesibilidad ni recuperación. Debe conservar el color actual, permanecer en Spark y ejecutar compilación y Android Lint después de cada cambio importante.

13 Cómo continuar el desarrollo sin perder contexto

Orden recomendado antes de modificar

1. Leer README.md y ROLES.md completos.

2. Revisar git status y el historial reciente para no borrar cambios del equipo.

3. Abrir Models.kt, AppViewModel.kt, DraftStore.kt, MunicipalRepository.kt, WorkRepository.kt y MainActivity.kt.

4. Distinguir entre funciones existentes, funciones simuladas y tareas todavía pendientes.

5. Diseñar el cambio y su impacto en reglas, datos, navegación, privacidad y Spark antes de escribir código.

6. Implementar en partes pequeñas, conservando los flujos anteriores.

7. Después de cada cambio importante ejecutar :app:assembleDebug y :app:lintDebug.

8. Si cambia Firebase, probar primero con emuladores y actualizar reglas, índices, pruebas y documentación.

Archivos de referencia

- README.md - alcance, versión, configuración, límites y verificaciones.
- ROLES.md - permisos, flujo municipal, privacidad y procedimiento de aprobación.
- tests/android-recovery.md - borradores y recuperación de envíos.
- tests/android-roles.md - recorrido administrador, veterinario y ciudadano.
- tests/android-events.md - propuesta, aprobación y publicación de eventos.
- tests/demo-second-cut.md - guion reproducible de demostración.
- firestore.rules y firestore.indexes.json - seguridad y consultas de Firestore.
- Bitacora_Avances_Mascotas_Municipales.docx - historia resumida del proyecto.
- Investigacion_Zipaquira_Mascotas_Municipales...docx - investigación territorial, legal y de accesibilidad.
- Taller_Offline_First_Firebase_RTDB_Desarrollo_Movil_2026_2.docx - nuevo requerimiento de RTDB manual.

14 Texto listo para enviar a otro chat

El siguiente texto puede copiarse junto con este documento cuando un compañero abra una conversación nueva:

> *Estoy trabajando en la aplicación Android Kotlin Mascotas Municipales, versión 1.5. Antes de proponer cambios, revisa este documento y los archivos reales del proyecto, especialmente README.md, ROLES.md y el historial de Git. La aplicación ya tiene Firebase Authentication y Cloud Firestore, flujos de ciudadano, administrador y veterinario, recuperación de borradores, accesibilidad, eventos territoriales aprobados, mapa OpenStreetMap y consulta QR. Debes conservar las funciones existentes, mantener el color actual, cuidar el rendimiento del emulador y mantener Firebase en Spark sin servicios pagos. No inventes que una función está terminada si aparece como simulada o pendiente. El siguiente trabajo es integrar en el mismo proyecto el taller Offline First con Room o persistencia equivalente y sincronización manual a Firebase Realtime Database. GUARDAR debe ser solo local y SINCRONIZAR debe enviar únicamente los pendientes sin duplicar. Después de cada cambio importante compila y ejecuta Android Lint, y actualiza las pruebas y documentación que correspondan.*

Cierre

Este documento debe mantenerse alineado con el código. Si cambia una función, una regla, una prueba o un límite importante, también debe actualizarse el contexto para que el siguiente integrante no trabaje sobre una versión antigua del proyecto.